



VIT[®]

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE204L- Design and Analysis of Algorithms

Dr. Iyappan Perumal

Assistant Professor Senior Grade 2

School of Computer Science & Engineering

VIT,Vellore.



Course Objective

1. To provide mathematical foundations for analysing the complexity of the algorithms
2. To impart the knowledge on various design strategies that can help in solving the real world problems effectively
3. To synthesize efficient algorithms in various engineering design situations



BCSE204L- Design and Analysis of Algorithms- Course outcome

1. Apply the mathematical tools to analyse and derive the running time of the algorithms
2. Demonstrate the major algorithm design paradigms.
3. Explain major graph algorithms, string matching and geometric algorithms along with their analysis.
4. Articulating Randomized Algorithms.
5. Explain the hardness of real-world problems with respect to algorithmic efficiency and learning to cope with it.



BCSE204L- Design and Analysis of Algorithms- Syllabus

Module:1	Design Paradigms: Greedy, Divide and Conquer Techniques	6 hours
Overview and Importance of Algorithms - Stages of algorithm development: Describing the problem, Identifying a suitable technique, Design of an algorithm, Derive Time Complexity, Proof of Correctness of the algorithm, Illustration of Design Stages - Greedy techniques: Fractional Knapsack Problem, and Huffman coding - Divide and Conquer: Maximum Subarray, Karatsuba faster integer multiplication algorithm.		
Module:2	Design Paradigms: Dynamic Programming, Backtracking and Branch & Bound Techniques	10 hours
Dynamic programming: Assembly Line Scheduling, Matrix Chain Multiplication, Longest Common Subsequence, 0-1 Knapsack, <u>TSP</u> - Backtracking: N-Queens problem, Subset Sum, Graph Coloring- Branch & Bound: LIFO-BB and FIFO BB methods: Job Selection problem, 0-1 Knapsack Problem		
Module:3	String Matching Algorithms	5 hours
Naïve String-matching Algorithms, KMP algorithm, Rabin-Karp Algorithm, Suffix Trees.		
Module:4	Graph Algorithms	6 hours
All pair shortest path: Bellman Ford Algorithm, Floyd-Warshall Algorithm - Network Flows: Flow Networks, Maximum Flows: Ford-Fulkerson, Edmond-Karp, Push Re-label Algorithm – Application of Max Flow to maximum matching problem		
Module:5	Geometric Algorithms	4 hours
Line Segments: Properties, Intersection, sweeping lines - Convex Hull finding algorithms: Graham's Scan, Jarvis' March Algorithm.		
Module:6	Randomized algorithms	5 hours
Randomized quick sort - The hiring problem - Finding the global Minimum Cut.		
Module:7	Classes of Complexity and Approximation Algorithms	7 hours
The Class P - The Class NP - Reducibility and NP-completeness – SAT (Problem Definition and statement), 3SAT, Independent Set, Clique, Approximation Algorithm – Vertex Cover, Set Cover and Travelling salesman		
Module:8	Contemporary Issues	2 hours

BCSE204L- Design and Analysis of Algorithms- References

Text Books:

1. Thomas H. Cormen, C.E. Leiserson, R L. Rivest and C. Stein, Introduction to Algorithms, 2009, 3rd Edition, MIT Press.

Reference Books:

1. Jon Kleinberg, Éva Tardos ,Algorithm Design, Pearson education, 2014.
2. Rajeev Motwani, Prabhakar Raghavan; Randomized Algorithms, Cambridge University Press, 1995 (Online Print – 2013)
3. Ravindra K.Ahuja, Thomas L.Magnanti and James B. Orlin, “ Network Flows: Theory, Algorithms and Applications”, Pearson Education, 2014.



Module 2: Design Paradigms: Dynamic Programming-Backtracking- Branch and Bound(10 Hours)

- **Dynamic Programming**
 - Matrix chain Multiplication
 - Assembly line Scheduling
 - Longest Common Subsequence
 - 0/1 Knapsack Problem
 - **TSP-Travelling Salesman Problem**
- **Backtracking**
 - N-queens Problem
 - Sum of Subsets
 - Graph Coloring
- **Branch and Bound**
 - LIFO and FIFO Methods
 - Job Selection Problem
 - 0/1 Knapsack Problem



Dynamic Programming

Dynamic programming is an algorithm design method that can be used when the solution to a problem can be viewed as the result of a sequence of decisions.

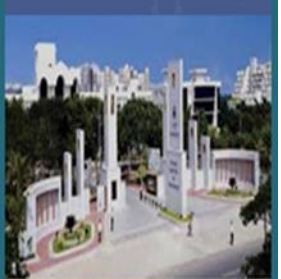
Principle of Optimality:

The principle of optimality states that an optimal sequence of decisions has the property that whatever the initial state and decision are, the remaining decisions must constitute an optimal decision sequence with regard to the state resulting from the first decision.



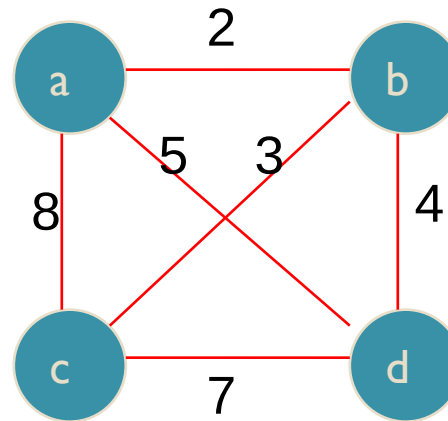
Travelling Salesman Problem

- Kind of Permutation Problem
- Permutation problems will usually be much harder to solve than subset problems as there are $n!$ different permutations of n objects while there are only 2^n different subsets of n objects ($n! > O(2^n)$).



Travelling Salesman Problem

- Given n cities with known distances between each pair, find the shortest tour that passes through all the cities exactly once before returning to the starting city.
- Consider the given graph:



Travelling Salesman Problem

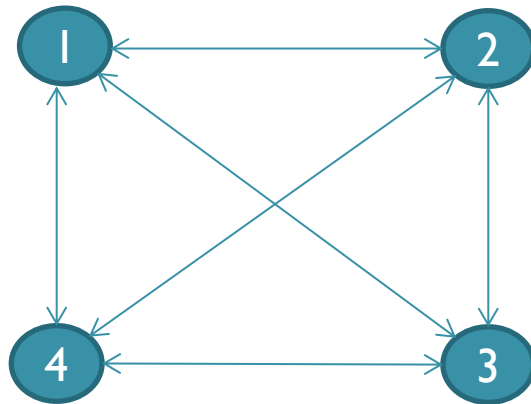
Path	Path Cost
$a \rightarrow b \rightarrow c \rightarrow d \rightarrow a$	$2+3+7+5 = 17$
$a \rightarrow b \rightarrow d \rightarrow c \rightarrow a$	$2+4+7+8 = 21$
$a \rightarrow c \rightarrow b \rightarrow d \rightarrow a$	$8+3+4+5 = 20$
$a \rightarrow c \rightarrow d \rightarrow b \rightarrow a$	$8+7+4+2 = 21$
$a \rightarrow d \rightarrow b \rightarrow c \rightarrow a$	$5+4+3+8 = 20$
$a \rightarrow d \rightarrow c \rightarrow b \rightarrow a$	$5+7+3+2 = 17$

- Have we considered all tours?
- Do we need to consider more?
- Any way to consider fewer?
- $O(n!)$ - in Brute Force Technique



Travelling Salesman Problem- Dynamic Programming

- Given n cities with known distances between each pair, find the shortest tour that passes through all the cities exactly once before returning to the starting city



$G=\{V,E\}$

	1	2	3	4
1	0	12	17	22
2	7	0	11	12
3	9	15	0	14
4	10	10	11	0

Adjacency Matrix



Travelling Salesman Problem- Dynamic Programming

- From the Principle of Optimality Rule:

$$g(1, V - \{1\}) = \min_{2 \leq k \leq n} \{c_{1k} + g(k, V - \{1, k\})\}$$

- Generalizing the above equation we obtain

$$g(i, S) = \min_{j \in S} \{c_{ij} + g(j, S - \{j\})\}$$



Travelling Salesman Problem- Dynamic Programming

- $G\{1, \{2,3,4\}\} = \min_{k \in \{2,3,4\}} \{C_{1k} + G\{k, \{2,3,4\}\} - \{k\}\}$

$$k \in \{2,3,4\}$$

- In general

- $G\{i, s\} = \min_{k \in s} \{C_{ik} + G\{k, s\} - \{k\}\}$

$$k \in s$$





Thank You